

Widget Decision Architecture: Who Decides What to Show?

Quick Answer

In your current project, you're using Tool-Driven (Deterministic):

```
// Agent calls SearchPostsTool
SearchPostsTool._call(input)

// Tool decides: ALWAYS returns card
return JSON.stringify({
  createSurface: {...},
  updateComponents: [{
    type: "card",    // ← TOOL DECIDED
    properties: {
      title: post.title,
      body: post.body
    }
  }]
})

// Agent ONLY passes to frontend
// Agent doesn't choose the widget
```

The 3 Architectures

1 ■■ TOOL-DRIVEN (Deterministic)

Concept: The tool returns complete A2UI. Agent only orchestrates which tool to call.

Flow:

```
User Query
↓
Agent recognizes intent
↓
"I should call SearchPostsTool"
↓
SearchPostsTool.invoke(input)
↓
```

Tool Returns: Complete A2UI

```
{
  "createSurface": {...},
  "updateComponents": [{
    "type": "card",          ← TOOL DECIDED!
    "properties": {
      "title": "${/post/title}",
      "body": "${/post/body}"
    }
  ]
}
```

↓

Agent passes to frontend

↓

Frontend renders

Implementation (Current in Your Project):

```
class SearchPostsTool extends Tool {
  name = "search_posts";

  async _call(input) {
    const post = await fetchPost(input.postId);

    // TOOL DECIDES: Always returns card
    return JSON.stringify({
      createSurface: {...},
      updateComponents: [{
        id: "post-card",
        type: "card",          // ← DECIDED HERE IN TOOL
        properties: {
          title: post.title,
          content: post.body
        }
      ]
    });
  }
}
```

Agent: "Tool returned A2UI, I'll pass it to frontend"

Agent: Takes no visual decision!

Advantages:

- ■ Predictable and consistent
- ■ Same input = same widget
- ■ Easy to test
- ■ Better performance (no extra analysis)
- ■ Fewer LLM tokens

Disadvantages:

- ■ Rigid - same response always
- ■ Doesn't adapt to context
- ■ Limited creativity
- ■ Overkill for complex data

When to Use:

- Simple CRUD (search, list, edit)
- Predictable operations
- Always same result type
- Performance is critical

2 ■ ■ AGENT-DRIVEN (Generative)

Concept: Tool returns raw data. Agent uses LLM to choose which A2UI to generate based on context.

Flow:

```
User Query: "Show posts in an interesting way"
↓
Agent (with LLM)
↓
"I'll call SearchPostsTool then generate A2UI"
↓
SearchPostsTool.invoke(input)
↓
Tool Returns: Raw Data
{
  "posts": [
    { "id": 1, "title": "Post 1", "body": "..." },
    { "id": 2, "title": "Post 2", "body": "..." }
  ]
}
↓
Agent (LLM analyzes context)
↓
"User wants 'interesting way'
I'll return a carousel instead of list"
↓
Agent Generates A2UI
{
  "type": "carousel",           ← AGENT DECIDED!
  "items": "${/posts}"
}
↓
Agent passes to frontend
↓
Frontend renders
```

Implementation:

```
class SearchPostsTool extends Tool {
  name = "search_posts";

  async _call(input) {
    const posts = await fetchPosts(input.query);

    // TOOL RETURNS: Only raw data
    return JSON.stringify({
      posts: posts,
      totalCount: posts.length
    });
  }
}

// AGENT PROCESSES
Agent SystemPrompt:
  "You received posts data. Decide which A2UI to use:
  - For 1-3 posts: card
  - For 4-10 posts: list
  - For 11+ posts: table with pagination
  - If user asked for 'interesting': carousel"

Agent Generates A2UI:
{
  "type": "carousel", // ← AGENT CHOSE!
  "items": "${/posts}"
}
```

Advantages:

- ■ Creative and adaptive
- ■ Responds to user context
- ■ Flexible for varied results
- ■ Leverages LLM capability

Disadvantages:

- ■ Unpredictable - results can vary
- ■ More tokens (LLM needs to process)
- ■ Slower
- ■ May generate invalid A2UI
- ■ Hard to test and debug

When to Use:

- Complex cases needing creativity

- UX varies significantly by context
- Natural conversations where tone changes
- Performance is not critical

3 ■ ■ HYBRID (Best of Both Worlds)

Concept: Tool returns data + UI suggestion. Agent can accept or modify based on context.

Flow:

```

User Query
↓
Agent calls Tool
↓
Tool Returns:
{
  "data": [...],
  "suggestedUIType": "list", ← TOOL SUGGESTS
  "context": "showing results"
}
↓
Agent (with system prompt)
↓
"Tool suggested 'list'. Context allows?
User asked for something specific?
I'll generate A2UI based on suggestion"
↓
Agent Generates A2UI
{
  "type": "list", ← AGENT CONFIRMED/MODIFIED
  "items": "${/data}"
}
↓
Frontend renders

```

Implementation:

```

class SearchPostsTool extends Tool {
  async _call(input) {
    const posts = await fetchPosts(input.query);

    return JSON.stringify({
      posts: posts,
      metadata: {
        count: posts.length,
        suggestedUI: posts.length > 10 ? "table" : "list"
        // ↑ TOOL SUGGESTS, but agent decides
      }
    });
  }
}

```

```
}
```

Agent SystemPrompt:

- ```
"1. Tool suggests UI type
2. You can accept or change
3. Consider:
 - Tool's suggestion
 - Conversation context
 - User's request
4. Generate appropriate A2UI"
```

Agent Decision:

```
if (userAsked("show creatively")) {
 return carousel; // ← Modified suggestion
} else {
 return suggestedUI; // ← Accepted suggestion
}
```

Advantages:

- ■ Predictable (has suggestion)
- ■ Flexible (can change)
- ■ Efficient (less analysis)
- ■ Robust (fewer errors)

Disadvantages:

- ■ More complex to implement
- ■ Needs tool ↔ agent communication

When to Use:

- Most real-world cases
- Balance between performance and flexibility
- Production

## Comparison Table

|                |                      |  |                    |  |                               |  |
|----------------|----------------------|--|--------------------|--|-------------------------------|--|
| Who Decides?   | Tool (deterministic) |  | Agent (generative) |  | Tool proposes, Agent confirms |  |
| Predictability | ■■■■■■■              |  | ■■■                |  | ■■■■■                         |  |
| Flexibility    | ■■■                  |  | ■■■■■■■            |  | ■■■■■                         |  |
| Performance    | ■■■■■■■              |  | ■■■                |  | ■■■■■                         |  |
| Complexity     | Low                  |  | High               |  | Medium                        |  |

| Use Case | Simple CRUD | Complex conversations | Production (Recommended) |  
| LLM Tokens | Few | Many | Moderate |

## Concrete Examples

### Scenario: "Show my posts"

Tool-Driven:

Tool: "I always return list"

Result:

```
{
 "type": "list",
 "items": "${/posts}"
}
```

Agent-Driven:

Agent: "Has 2 posts, I'll  
use simple card"

OR

Agent: "Has 50 posts,  
I'll use table"

Hybrid:

Tool: "I suggest list"

Agent: "Tool suggested list,  
context OK, I'll use list"

## Production Recommendation

Use HYBRID:

- Tools return data + `suggestedUIType`
- System prompt guides agent decision
- Agent respects suggestion (performance)

- Agent can override if context demands (flexibility)

Benefit: 80% performance of tool-driven, 80% flexibility of agent-driven!

## In Your Project

Currently you're using Tool-Driven:

SearchPostsTool always returns:

```
{
 "type": "card",
 "properties": {
 "title": "${/post/title}",
 ...
 }
}
```

UpdatePostTool always returns:

```
{
 "type": "alert",
 "severity": "success"
}
```

## To switch to Hybrid:

Tool returns:

```
{
 "post": {...},
 "suggestedUI": "card"
}
```

System Prompt:

1. If suggestedUI exists, use it
2. If context asks for something different, change as needed
3. Generate appropriate A2UI"

Agent Processes:

```
"Tool suggested card, I'll use it"
OR
"User asked for 'creative', I'll use carousel"
```



## Next Steps

- Understand which approach your case needs
- Tool-Driven: Simple, fast
- Hybrid: Recommended for production
- Agent-Driven: When you need extreme creativity

## Final Summary

The decision of which widget to show can come from 3 places:

- **■ Tool** (deterministic) - "SearchPostsTool always returns card"
- **■ Agent** (generative) - "Agent analyzes context and chooses"
- **■ Hybrid** (recommended) - "Tool suggests, agent confirms/modifies"

The choice depends on your requirements for performance, flexibility, and complexity!

Version: A2UI v1.0

Langchain: v0.1.24+